



GRIFFITH COLLEGE DUBLIN

Student name:	Lucas Rodrigues Crema					
Student number:	3133143					
Faculty:	Computing Science					
Course:	BSCH			Stage/year:	2	
Subject:	Software Development 2					
Study Mode:	Full time	☒		Part-time		
Lecturer Name:	Gemma Deery					
Assignment Title:	Report 3					
Date due:	30/04/2025					
Date submitted:	30/04/2025					

Plagiarism disclaimer:

I understand that plagiarism is a serious offence and have read and understood the college policy on plagiarism. I also understand that I may receive a mark of zero if I have not identified and properly attributed sources which have been used, referred to, or have in any way influenced the preparation of this assignment, or if I have knowingly allowed others to plagiarise my work in this way.

I hereby certify that this assignment is my own work, based on my personal study and/or research, and that I have acknowledged all material and sources used in its preparation. I also certify that the assignment has not previously been submitted for assessment and that I have not copied in part or whole or otherwise plagiarised the work of anyone else, including other students.

Signed: Lucas Rodrigues Crema

Date: 30/04/2025

Please note: Students MUST retain a hard / soft copy of ALL assignments as well as a receipt issued and signed by a member of Faculty as proof of submission.

Table of Contents

<i>Updating Sprites</i>	3
UnitTest	4
Main challenges	4
<i>Fixing Errors</i>	5
<i>Bibliography</i>	6

Updating Sprites

For this review I mostly worked on making it so the sprites for players changed accordingly with the direction they were moving.

```
public Player( String fileName,int x , int y, int width, int height ) {
    super(fileName, x,y,width,height);

    //Setting proper sprites for players
    try {
        //First, check if its player1 or player2, then set appropriate file (following same logic as in the sprite class constructor
        imageIdle = ImageIO.read(new File ("images/" + (fileName.contains("player1") ? "player1_idle.png" : "player2_idle.png")));
        imageLeft = ImageIO.read(new File ("images/" + (fileName.contains("player1") ? "player1_left.png" : "player2_left.png")));
        imageRight = ImageIO.read(new File ("images/" + (fileName.contains("player1") ? "player1_right.png" : "player2_right.png")));
        imageUp = ImageIO.read(new File ("images/" + (fileName.contains("player1") ? "player1_up.png" : "player2_up.png")));

        setImage(imageIdle);
    } catch (IOException e) {
        e.printStackTrace();
        System.out.println("file not found");
    }
}
```

First, I changed the constructor for Player and stored the corresponding images in separate BufferedImage and wrapped it around a try catch block in case of any IOException (overall its very similar to how we previously handled loading images as shown in the Sprite constructor in a different class below:

```
public Sprite(String fileName , int x, int y , int width, int height) {
    this.x = x;
    this.y = y - height;
    this.width = width;
    this.height = height;
    File pic = new File("images/"+fileName);
    try {
        image = ImageIO.read(pic);
    } catch (IOException e) {
        e.printStackTrace();
        System.out.print("file not found");
    }
}
```

Then, for every method related to movement (still on Player class), added setImage() which changes the sprite to the appropriate one depending on the action being taken.

```
public void moveRight()
{
    int xMax = Constants.SCREEN_SIZE.width - Constants.PLAYER_WIDTH;
    if(getX() + getWidth() < xMax) {
        this.setX(getX() + Constants.PLAYER_SPEED);
        setImage(imageRight);
    }
}

public void moveLeft()
{
    int xMin = 0;
    if(getX() - Constants.PLAYER_SPEED > xMin) { // don't go off screen
        this.setX(getX() - Constants.PLAYER_SPEED);
        setImage(imageLeft);
    }
}

public void jump()
{
    if(jumping == false)
    {
        setY(getY() - Constants.PLAYER_JUMP_HEIGHT);
        jumping = true;
        setImage(imageUp);
    }
}
```

On the GameManager class on the existing keyReleased() I also added a way to tell whether the player is currently idle or not, and to update the sprite accordingly.

```
public void keyReleased(int code) {
    activeKeys.remove(code); //Removing key pressed from HashSet once released
    updatePlayerMovement();

    //If no keys are being pressed, set player's image to idle
    if (!activeKeys.contains(Constants.LEFTP1) && !activeKeys.contains(Constants.RIGHTP1) && !activeKeys.contains(Constants.UPP1)) {
        player.setImage(player.getImageIdle());
    }
    if (!activeKeys.contains(Constants.LEFTP2) && !activeKeys.contains(Constants.RIGHTP2) && !activeKeys.contains(Constants.UPP2)) {
        player2.setImage(player2.getImageIdle());
    }
}
```

UnitTest

Added a unitTest for the store to ensure all of its parameters are correct

```
@Test
void testStore() {
    GameManager game = new GameManager();
    Store store = game.getStore();

    //Testing if the stores parameters are consistent
    assertEquals("bank.png", store.getFileName());
    assertEquals(1000, store.getX());
    assertEquals(Constants.GROUND_HEIGHT, store.getY());
    assertEquals(Constants.STORE_WIDTH, store.getWidth());
    assertEquals(Constants.STORE_HEIGHT, store.getHeight());
}
```

Main challenges

The main challenge I had was to figure out a way to correctly assign each sprite to the correct place, to deal with this I just created fields specific for each sprite, and assigned them accordingly depending on the action being taken.

Fixing Errors

Once the split screen feature was added, I had to change the logic behind how the store is drawn on the screen. This consisted of adding cameraX1 (which calculates the X value for the first screen) and storeScreenX (which calculates where the middle of said screen is) as shown below (same process for drawSprites2()).

```
public void drawSprites1(Graphics2D graphics, JPanel panel) {  
    //Draw player  
    graphics.drawImage(player.getImage(), player.getX(), player.getY(), player.getWidth(), player.getHeight(), panel);  
  
    //Calculating the X value for the first screen  
    int cameraX1 = player.getX() - Constants.SCREEN_WIDTH / 4;  
    //The middle X value of said screen  
    int storeScreenX = store.getX() - cameraX1;  
    graphics.drawImage(store.getImage(), storeScreenX, store.getY(), store.getWidth(), store.getHeight(), panel);  
  
    //Draw blocks  
    ArrayList<Block> tempBlocks = new ArrayList<>(blocks); // Make a copy of the blocks list  
    synchronized(blocks) {  
        for (Block block : tempBlocks) {  
            graphics.drawImage(block.getImage(), block.getX(), block.getY(), block.getWidth(), block.getHeight(), panel);  
        }  
    }  
  
    //Draw GUI - score  
    graphics.setColor(Color.white);  
    graphics.setFont(Constants.SCORE_FONT);  
    graphics.drawString(Integer.toString(player.getScore()), 20, 20);  
  
    //Draw countdown timer  
    int minutes = timeLeft/60;  
    int seconds = timeLeft%60;  
    graphics.setColor(Color.RED);  
    graphics.setFont(Constants.SCORE_FONT);  
    graphics.drawString(String.format("Time Left: %02d:%02d", minutes, seconds), 20, 50);  
}
```

Bibliography

RyiSnow. (2021, 10 12). *Sprites and Animation - How to Make a 2D Game in Java #3*.

Retrieved from Youtube: <https://www.youtube.com/watch?v=wT9uNGzMEM4>